

gemstone-rs VS Code Workbench

Sidebar browsing, codegen, and explorer launch workflows



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

VS Code Workbench

Setup

Sidebar Tree

Command Palette

Codegen Workflow

Object Mapping Workflow

Embedded Explorer Webview

Develop the Extension

Later Feature Work

VS Code Workbench

`gemstone-rs Workbench` is the VS Code companion for the Rust CLI and local explorer. It keeps the CLI as the stable contract and adds a sidebar, output panel, generated-file previews, and codegen diff prompts.

Marketplace:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup

For installed CLI tools:

```
{
  "gemstoneRs.cliPath": "gemstone-rs",
  "gemstoneRs.explorerPath": "gemstone-rs-explorer",
  "gemstoneRs.codegenConfig": "gemstone-rs.codegen"
}
```

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Use the same `GS_*` environment as the CLI.

Sidebar Tree

Open the GemStone RS activity bar item.

The sidebar contains:

- `Dictionaries`
- `Codegen Config`
- `Explorer`

Dictionaries expands live dictionaries, classes, protocols, and methods. Selecting a method opens its source in an untitled Smalltalk editor.

Codegen Config exposes:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- List BridgeRoot Keys
- Put BridgeRoot String
- Remove BridgeRoot Key
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles
- Save Project Profiles
- Export Codegen Profile
- Show Sample Project Profiles
- Create Project Profiles
- Validate Project Profiles
- Open Codegen Docs

Explorer exposes:

- Doctor
- Verify Setup
- Eval Smalltalk
- Launch Explorer
- Open Explorer Webview

Command Palette

Commands:

- GemStone RS: Verify Setup
- GemStone RS: Doctor
- GemStone RS: Eval Smalltalk
- GemStone RS: Browse Dictionaries
- GemStone RS: Browse Classes
- GemStone RS: Codegen Init
- GemStone RS: Codegen Discover
- GemStone RS: Generate Mapping Config
- GemStone RS: Preview BridgeRoot
- GemStone RS: List BridgeRoot Keys

- GemStone RS: Put BridgeRoot String
- GemStone RS: Remove BridgeRoot Key
- GemStone RS: Run Generated Mapping Example
- GemStone RS: Codegen Preview
- GemStone RS: Codegen Diff
- GemStone RS: Codegen Check
- GemStone RS: Codegen Generate
- GemStone RS: Load Project Profiles
- GemStone RS: Save Project Profiles
- GemStone RS: Export Codegen Profile
- GemStone RS: Show Sample Project Profiles
- GemStone RS: Create Project Profiles
- GemStone RS: Validate Project Profiles
- GemStone RS: Launch Explorer
- GemStone RS: Open Explorer Webview
- GemStone RS: Open Method Source
- GemStone RS: Open Codegen Docs

GemStone RS: Verify Setup and GemStone RS: Doctor both run the CLI `gemstone-rs doctor` report, so the workbench setup view and terminal setup use the same diagnostic path.

Codegen Workflow

1. Set `gemstoneRs.codegenConfig`.
2. Run GemStone RS: Codegen Discover or edit the config manually.
3. Run GemStone RS: Codegen Preview.
4. Run GemStone RS: Codegen Diff.
5. Run GemStone RS: Codegen Generate.

Codegen Generate runs the diff first. If output would change, it opens the diff and asks before writing.

Object Mapping Workflow

Use GemStone RS: Generate Mapping Config when you want the live stone to inspect a GemStone class and propose a BridgeMapped config. The command asks for:

- config path
- Rust struct name
- GemStone class reference, such as `Object` or `UserGlobals:0kzBooking`

Use GemStone RS: Preview BridgeRoot after starting the local explorer. It opens:

```
http://127.0.0.1:8787/api/bridge/root
```

Use `GemStone RS: List BridgeRoot Keys` for the same inspection path without opening a browser; it runs `gemstone-rs bridge keys` and writes the key OOPs, class OOPs, `printString` values, and identity ids to the output panel.

Use `GemStone RS: Put BridgeRoot String` when you want a quick live-write smoke test from VS Code. It asks for a key and string value, then runs:

```
gemstone-rs bridge put <key> <value> --type String
```

Use `GemStone RS: Remove BridgeRoot Key` to remove one BridgeRoot key after a confirmation prompt:

```
gemstone-rs bridge remove <key>
```

Use `GemStone RS: Run Generated Mapping Example` to run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Embedded Explorer Webview

Use `GemStone RS: Launch Explorer` first. It starts `gemstone-rs-explorer` in a terminal and opens the browser. Then use `GemStone RS: Open Explorer Webview` to embed the local explorer inside VS Code.

The webview points at:

```
http://127.0.0.1:8787/
```

It is still the same loopback-only explorer process. If the explorer is not running, the webview will show a connection error and the output panel will show the URL to start.

The embedded page includes the same browse, BridgeRoot, and Codegen Workflow panels as the external browser view. That makes it the quickest place to preview generated wrappers, inspect unified and side-by-side diffs, run `codegen check`, and test BridgeRoot key/value policies without leaving VS Code. The explorer page can load and save the selected codegen config file, posting the editor contents as the request body; saves still require the explorer to run with `--allow-write`. Use `Refresh Configs` in the embedded page to populate the `Known configs` picker from the explorer process working tree, or set `Config root` in the page when the explorer was launched from a different directory. The page also keeps a local `Recent configs` picker for paths used by

Load, Save, Preview, Diff, Check, and Generate. Use `Save Profile` when you want to switch between named codegen workflows; a profile stores the config root, config path, mapped Rust type, and GemStone class in local browser storage. `Export Profile` writes a JSON payload into `Profile JSON`, and `Import Profile` merges pasted profile JSON from another browser or teammate. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the configured codegen root, and `Save Project Profiles` writes the saved profile list back when the explorer was launched with `--allow-write`. Project profile payloads are schema-validated before writing, and imports report which profiles are new, replaced, or unchanged. It remembers its current fields locally, so repeated codegen or BridgeRoot checks keep the same config path and class selection.

Use `GemStone RS: Show Sample Project Profiles` to open the built-in sample from `gemstone-rs profile sample`, `GemStone RS: Create Project Profiles` to write that sample into a project file with `gemstone-rs profile init`, and `GemStone RS: Validate Project Profiles` when you want a direct CLI check without opening the explorer. Validation runs `gemstone-rs profile validate` against the selected profile file and writes the result to the GemStone RS output panel. The extension also contributes JSON validation for files named `gemstone-rs.codegen-profiles.json`.

Develop the Extension

```
cd vscode-gemstone-rs-workbench
npm ci
npm run check
npm run test:smoke
npm run package -- --out gemstone-rs-workbench-0.3.0.vsix
```

From the repository root:

```
make vscode-package
```

Later Feature Work

The first embedded webview is available. Next work should make the webview more IDE-like: generate wrapper diffs before writing, edit codegen selections, inspect BridgeRoot payloads visually, and keep browser fallback behavior for users who prefer the external explorer.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.